

# Overview of Research Computing: ITS HPC Clusters and National Resources

for CSC4760/5760

Sharon Colson   Mike Renfro

Information Technology Services

2026-02-04

## Mike's Intro

- ▶ Three-time Tech mechanical engineering graduate
- ▶ Supporting computing for science and engineering research and education since 1991
- ▶ Full-time HPC systems administrator since 2017
- ▶ Director of Research Computing and Data as of August 2025

### Member/contributor for:

- ▶ Campus Champions, formerly part of NSF's Extreme Science and Engineering Discovery Environment (XSEDE)
- ▶ NSF ACCESS (formerly XSEDE) Cyberinfrastructure Resource Integration (CRI) group
- ▶ ACM SIGHPC Systems Professionals
- ▶ Campus Research Computing Consortium (CaRCC)
- ▶ Instructor for Software Carpentry / HPC Carpentry

## Sharon's Intro

- ▶ Senior Research Technology Engineer
- ▶ 2025 CS graduate (Data Science and AI concentration)
- ▶ Dual minors in humanities and science (focused on Chemistry)
- ▶ Former NSF ACCESS Intern (AtlanticWave-SDX project)

Member/contributor for:

- ▶ Campus Champions
- ▶ Instructor for Software Carpentry / HPC Carpentry
- ▶ ACM

## Trevor's Intro

- ▶ Student Research Computing and Data Facilitator
- ▶ Junior CS major (HPC concentration)
- ▶ Little bit of experience in Windows systems administration
- ▶ Just started in April 2025

## What We Do (not just running the local HPC)

Campus Research Computing Consortium's "5 Facings":

**Researcher-Facing** user support/training/onboarding

**Data-Facing** data creation/discovery/collection, data analysis/vizualization

**Software-Facing** software installation, troubleshooting, workflow engineering, containers

**Systems-Facing** hardware infrastructure, systems administration, security

**Strategy/Policy-Facing** leadership, institutional alignment, partnership/engagement  
outside the university

Some of these extend to national resources, including NSF and DOE facilities.

## A Whirlwind Tour of National Facilities

NSF ACCESS <https://access-ci.org>

- ▶ Four levels of projects, with the lowest level requiring a paragraph and sometimes granted next-day
- ▶ Zero charge for resources, including traditional HPC, novel/composable hardware, and private cloud
- ▶ Also offers training/help/consulting, sometimes for free

DOE ASCR <https://science.osti.gov/ascr>

- ▶ Larger projects
- ▶ Broadening the HPC community
- ▶ Projects aligning with DOE mission

## A Whirlwind Tour of National Facilities

National AI Research Resource (NAIRR) Pilot <https://nairrpilot.org>

- ▶ Joint program among NSF and other federal agencies
- ▶ AI-focused facilities:
  - ▶ some are literally NSF ACCESS, DOE ASCR, and other federally funded facilities
  - ▶ others are commercial offerings (Cerebras accelerators, SambaNova inference, etc.)

## Original Dell Cluster Purchase (just under \$400k in 2016)

### 44 Compute Nodes

- ▶ Dual Xeon, 28 2.4 GHz cores per node
- ▶ 64-896 GB RAM per node
- ▶ 250-300 GB disk space per node
- ▶ 2 nodes include dual NVIDIA K80 GPUs (up to 8 GPU jobs at once)
- ▶ 1232 cores and 5 TB RAM total

### Infrastructure

- ▶ 240 TB file server (all hard disk, 180 TB usable, now retired)
- ▶ Internal 56 Gb Infiniband network
- ▶ Backup and long-term file storage in progress



## Later Purchases (around \$900k from 2021-2024)

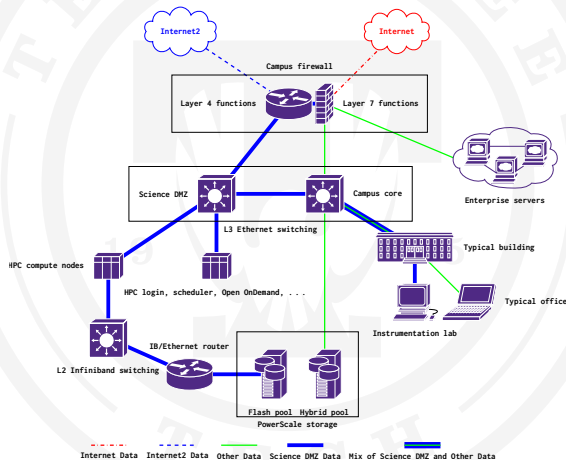
### 10 Compute Nodes

- ▶ Dual AMD Epyc3, 128 2.4 GHz cores per node
- ▶ 512 GB RAM per node
- ▶ 3500 GB SSD space per node
- ▶ Dual NVIDIA A100 (40 GB) GPUs
- ▶ 1280 cores and 5 TB RAM total

### Infrastructure

- ▶ 300 TB SSD file server
- ▶ Internal 100 Gb Infiniband network

# The Architecture



## The Software

- ▶ Rocky Linux 8 on login node and compute nodes
- ▶ spack system for accessing (most) cluster-wide software installations
  - ▶ Artificial Intelligence/Deep Learning/Machine Learning: TensorFlow, PyTorch, Keras, scikit-learn
  - ▶ Biology: BLAST+, Mothur, RAxML, STACKS
  - ▶ Chemistry: Gaussian, Gromacs, LAMMPS, NAMD
  - ▶ FEA/CFD: ANSYS (including EM and Fluent), COMSOL
  - ▶ Math: MATLAB, R (plus lots of R modules)
  - ▶ Software development: CUDA, FFTW, multiple parallel and linear algebra libraries
- ▶ Use conda and virtualenv to manage your own installation of Python and other tools
- ▶ Use singularity to manage your own Linux containers

## Secure Shell

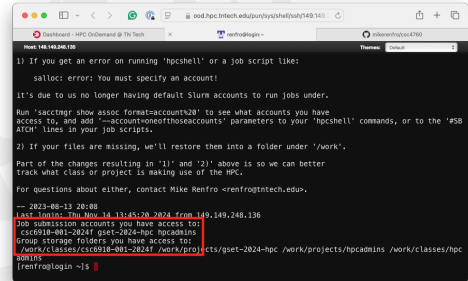
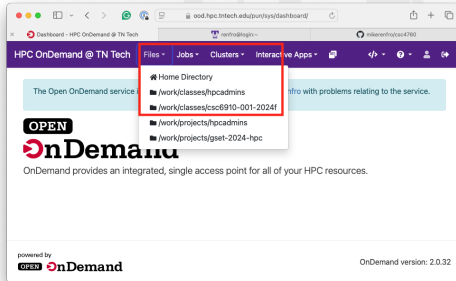
On campus or with the campus VPN, SSH to `login.hpc.tntech.edu` to access HPC.  
**Use your TTU username** from your personal laptop, else your IP address will probably get blocked for a few minutes.

 renfro@login:~

```
PS C:\Users\Renfro> ssh renfro@login.hpc.tntech.edu
The authenticity of host 'login.hpc.tntech.edu (149.149.248.135)' can't be established.
ECDSA key fingerprint is SHA256:JJiFgi6rvJQil5U1iSeT92WgMqAguFBN4ZkwhZwBMWg.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'login.hpc.tntech.edu,149.149.248.135' (ECDSA) to the list of known hosts.
renfro@login.hpc.tntech.edu's password:
Last failed login: Tue Jan 16 19:57:06 CST 2024 from 149.149.17.45 on ssh:notty
There was 1 failed login attempt since the last successful login.
Last login: Tue Jan 16 19:53:59 2024 from 149.149.17.45
[renfro@login ~]$ srun --ntasks=100 --account=hpcadmins hostname | sort | uniq -c
    1 gpunode003
   46 gpunode004
   53 gpunode005
[renfro@login ~]$
```

## Open OnDemand <https://ood.hpc.tntech.edu/>

Includes web-based file manager, terminal, Jupyter notebooks, and works from off campus without VPN



## The Scheduling System (Slurm)

Queueing system schedules, reserves, and protects resources for jobs submitted from the login node with `sbatch`, `hpcshell`, or similar tools; or from HPC-enabled desktop applications like ANSYS Workbench.

```
renfro@login:~
PS C:\Users\Renfro> ssh renfro@login.hpc.tntech.edu
The authenticity of host 'login.hpc.tntech.edu (149.149.248.135)' can't be established.
ECDSA key fingerprint is SHA256:JJiFgi6rvJQil5U1iSeT92WgMqAguFBN4ZkwhZwBMWg.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'login.hpc.tntech.edu,149.149.248.135' (ECDSA) to the list of known hosts.
renfro@login.hpc.tntech.edu's password:
Last failed login: Tue Jan 16 19:57:06 CST 2024 from 149.149.17.45 on ssh:notty
There was 1 failed login attempt since the last successful login.
Last login: Tue Jan 16 19:53:59 2024 from 149.149.17.45
[renfro@login ~]$ srun --ntasks=100 --account=hpcadmins hostname | sort | uniq -c
    1 gpunode003
   46 gpunode004
   53 gpunode005
[renfro@login ~]$
```

## Scheduling Techniques

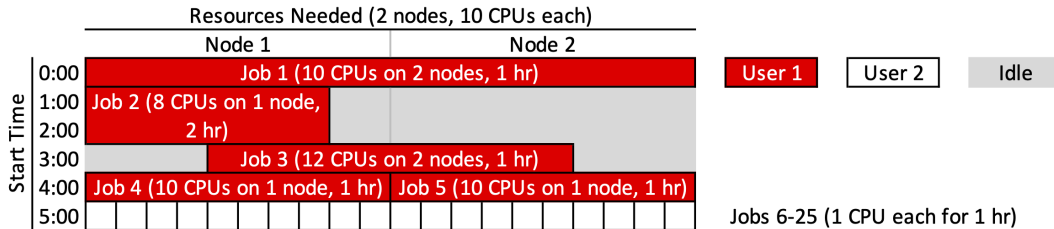
- ▶ Mostly first in, first out (FIFO)
- ▶ Backfill smaller, shorter jobs while larger jobs are waiting for enough resources
- ▶ Adjust queued job priorities to give all users a fair share of resources

The next few slides show how a set of two users' jobs could be scheduled. Goals:

- ▶ minimize idle resource time
- ▶ minimize time for jobs to complete

## Scheduling Example (Pure FIFO)

Pure FIFO strategy for 25 jobs submitted back to back

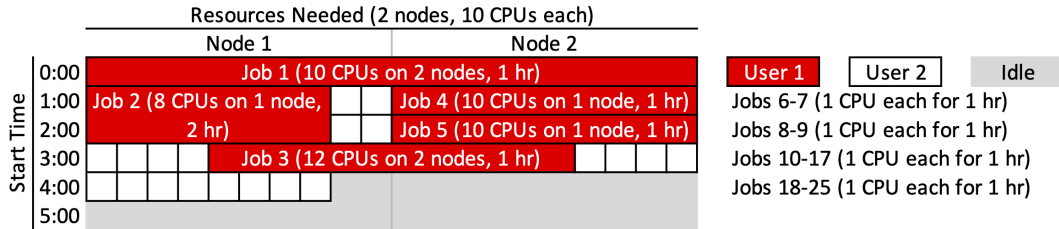


32 CPU hours idle with pending jobs, user 1 finishes in 5 hours, user 2 finishes in 6 hours (after waiting 5 hours for **anything** to start).



## Scheduling Example (FIFO with backfill)

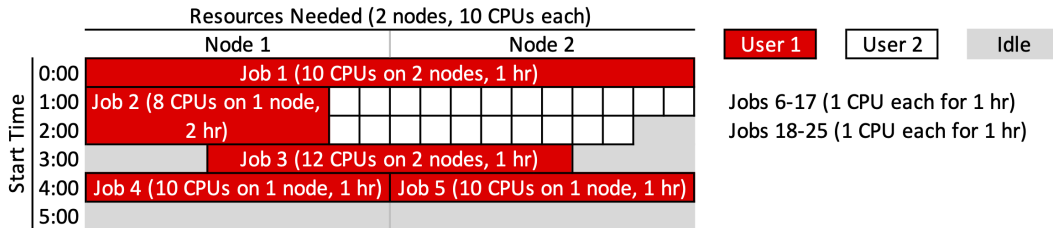
FIFO/backfill strategy for 25 jobs submitted back to back



0 CPU hours idle with pending jobs, user 1 finishes in 4 hours, user 2 finishes in 5 hours (only waiting 1 hour for jobs to start).

## Scheduling Example (FIFO with backfill and fair share job priorities)

FIFO/backfill/fairshare strategy for 25 jobs submitted back to back



10 CPU hours idle with pending jobs, user 1 finishes in 5 hours, user 2 finishes in 3 hours (only waiting 1 hour for jobs to start). If user 1 submitted job order was 1, 2, 4, 5, 3, would only have 2 CPU hours idle with pending jobs.

## Job Queues (Partitions in Slurm)

Queues try to segment different job types (by amount of resources and time needed), balancing the need to quickly develop and debug codes against the need to run larger production jobs

- ▶ batch-warp (default) and batch-impulse partitions for large jobs (max 7 days, low priority)
- ▶ debug partition for large, very short-running jobs (max 0.5 hours, medium priority)
- ▶ any-interactive partition for smaller, short-running jobs (max 2 hours, high priority)
- ▶ Other partitions exist (bigmem, hugemem), but most users typically only have to consider these three
- ▶ GPU partitions exist, but batch and interactive jobs are automatically routed there whenever GPUs are requested

## Primary Scheduler Policies

- ▶ Basically FIFO for equivalent resource requests
- ▶ Every job has a per-project or per-class account parameter to track usage (different than a username and password)
- ▶ When system usage is high enough to prevent new jobs from running immediately, jobs in accounts with heavy usage may queue behind jobs in accounts with light usage, to promote fair distribution of resources.

Reserve what you need (in terms of CPU, RAM, and GPUs), but use what you reserve!

Just because you reserve additional CPUs or a GPU doesn't automatically mean your program can effectively use either!

## How Much Time and Resources Should I Reserve for My Job?

- ▶ Everybody **wants** to start now
- ▶ Everybody **wants** to run fast
- ▶ Everybody **needs** to run successfully
- ▶ Everybody **should want** to have jobs finish sooner, regardless of when they start

## How Much Time and Resources Should I Reserve for My Job?

Is this job still in development and testing?

- ▶ Use `hpcshell` to test interactively and eliminate crashes on small problem sizes. Still have a 2-hour time limit, but can reserve multiple cores and nodes as needed.
- ▶ If you have to scale past the limits of the interactive partitions, make a job script for the debug partitions.
- ▶ Record your run times on different problem sizes and resource sizes with the `time` shell command or available functions in your language of choice.
- ▶ You can also `ssh` from the login node to a node you're running on, and use `top` or other tools to monitor performance in real time.

## How Much Time and Resources Should I Reserve for My Job?

Is this job debugged and ready for larger production runs?

- ▶ If you know Big O notation from analysis of algorithms, use that and your recorded run times to estimate time required.
- ▶ If you don't know Big O notation, most job run times scale as a function of problem size and core counts.
- ▶ If you've gathered run times from several small jobs, you should be able to curve fit those results to estimate time for your larger jobs using more cores.

Add a small amount of extra time (10%?) just in case. Also try to save intermediate results that you can resume from, rather than having to restart from scratch each time.

## File Management

- ▶ From **on campus or with VPN**, can scp, sftp, and rsync as `your_username@login.hpc.tntech.edu`.
- ▶ Some familiarity with a Linux command line and terminal-style text editors will be useful (see <http://linuxcommand.org/> for a good free introductory book)
- ▶ <https://ood.hpc.tntech.edu/> (Open OnDemand) works **both on and off campus without VPN**, includes a more graphical file manager

Your files could be stored in:

- ▶ Your home directory (`/home/tntech.edu/username`), up to 50 GB
- ▶ A class/project folder (`/work/projects/projectname/username`, `/work/classes/class-section-semester/username`), with a larger storage limit



## MPI/OpenMP Overview

**MPI** Message Passing Interface: parallel programming method where work is divided among multiple worker tasks (processes), and workers' results are transmitted to a coordinator task.

**OpenMP** Open Multi-Processing: parallel programming method where work is divided up among multiple threads in a single task, and threads' results are stored in a shared memory space.

- ▶ These methods can be combined into a hybrid scheme: one where worker tasks are multi-threaded via OpenMP, and transmit results back to a coordinator task via MPI.
- ▶ The following hybrid example uses 1 or more tasks, and 1 or more CPUs per task.

## Interactive Development and Debugging

Reserving 2 nodes, 3 tasks per node, 4 CPUs per task, downloading, compiling

```
[renfro@login ~]$ hpcshell -N 2 -n 6 -c 4 -A hpcadmins
# Normally I'd use --nodes=2 , --ntasks-per-node=3 (instead of
# --ntasks=6), --cpus-per-task=4, and
# --account=hpcadmins, but it takes up too much room on the
# slide
[renfro@gpunode009 ~]$ git clone \
> https://github.com/mikerenfro/csc4760
[renfro@gpunode009 ~]$ cd csc4760
[renfro@gpunode009 csc4760]$ spack load openmpi
[renfro@gpunode009 csc4760]$ make -f Makefile.mpi hw_omp_mpi
mpicc -fopenmp      hw_omp_mpi.c      -o hw_omp_mpi
```

## Interactive Development and Debugging

### Testing, releasing reservation

```
[renfro@gpunode009 csc4760]$ export \  
> OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}  
[renfro@gpunode009 csc4760]$ mpirun ./hw_omp_mpi \  
> | sort # sorting output for readability  
...  
I'm on node (compute server) gpunode010, rank (parallel  
process) 5 of 6, thread (CPU in process) 3 of 4  
[renfro@gpunode009 csc4760]$ exit  
exit  
[renfro@login ~]$
```

## Making job script for unattended batch run: contents of `hybrid.sh`

```
#!/bin/bash
# Running on 2 nodes
#SBATCH --nodes=2
# 3 tasks per node, 4 CPUs per task
#SBATCH --ntasks-per-node=3 --cpus-per-task=4
# Use CSC6910 job submission account
#SBATCH --account=csc4760-001-2026s
# run up to 1 minute
#SBATCH --time=00:01:00
spack load openmpi
export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}
mpirun ./hw_omp_mpi | sort
```

## Making job script for unattended batch run: creating `hybrid.sh`

- ▶ Start editing script with `nano hybrid.sh`
- ▶ Type the 12 lines shown on the previous slide
- ▶ Press Ctrl-X to prompt for save and exit

Submit job script with: `sbatch hybrid.sh` and look for `slurm-NNNNNN.out` file.

## Kokkos Overview

- ▶ Kokkos is an abstraction layer for HPC programming that supports multiple CPUs, GPUs, and other backends
- ▶ Hardware-agnostic by default

## Interactive Development and Debugging (without GPU)

Reserving 1 node, 1 tasks per node, 2 CPUs per task, downloading

```
[renfro@login ~]$ hpcshell -N 1 -n 1 -c 2 -A hpcadmins  
# Normally I'd use --nodes=1 , --ntasks-per-node=1 (instead of  
# --ntasks=1), --cpus-per-task=2, and  
# --account=hpcadmins, but it takes up too much room on the  
# slide  
[renfro@gpunode009 ~]$ git clone \  
> https://github.com/mikerenfro/csc4760  
[renfro@gpunode009 ~]$ cd csc4760/kokkos  
[renfro@gpunode009 kokkos]$ spack load kokkos@4.7~cuda gcc@14 \  
> cmake
```

## Interactive Development and Debugging (without GPU)

### Compiling

```
[renfro@gpunode009 kokkos]$ ls
01_hello_world  multi-gpu-cuda
[renfro@gpunode009 kokkos]$ cd 01_hello_world
[renfro@gpunode009 kokkos]$ mkdir build ; cd build
[renfro@gpunode009 build]$ cmake ..
-- The C compiler identification is GNU 14.3.0
...
[renfro@gpunode009 build]$ make
...
[100%] Linking CXX executable HelloWorld
[100%] Built target HelloWorld
```



## Interactive Development and Debugging (without GPU)

### Testing, releasing reservation

```
[renfro@gpunode009 build]$ export \  
> OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}  
[renfro@gpunode009 build]$ ./HelloWorld  
...  
Hello from i = 13  
Hello from i = 14  
[renfro@gpunode009 build]$ exit  
exit  
[renfro@login ~]$
```

## Interactive Development and Debugging (with GPU)

Reserving 1 node, 1 tasks per node, 2 CPUs per task, 1 GPU

```
[renfro@login ~]$ hpcshell -N 1 -n 1 -c 2 -A hpcadmins -G 1
# Normally I'd use --nodes=1 , --ntasks-per-node=1 (instead of
# --ntasks=1), --cpus-per-task=2, --account=hpcadmins, and
# --gpus=1, but it takes up too much room on the
# slide
[renfro@gpunode009 ~]$ cd ~/csc4760/kokkos
[renfro@gpunode009 kokkos]$ spack load kokkos@4.7+cuda gcc@14 \
> cmake
```

## Interactive Development and Debugging (with GPU)

### Compiling

```
[renfro@gpunode009 kokkos]$ ls
01_hello_world  multi-gpu-cuda
[renfro@gpunode009 kokkos]$ cd 01_hello_world
[renfro@gpunode009 kokkos]$ rm -rf build ; mkdir build ; cd build
[renfro@gpunode009 build]$ cmake ..
-- The C compiler identification is GNU 14.3.0
...
[renfro@gpunode009 build]$ make
...
[100%] Linking CXX executable HelloWorld
[100%] Built target HelloWorld
```

## Interactive Development and Debugging (with GPU)

### Testing

```
[renfro@gpunode009 build]$ export \  
> OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}  
[renfro@gpunode009 build]$ ./HelloWorld  
...  
Hello from i = 13  
Hello from i = 14
```

## Example that actually uses a GPU

### Compiling

```
[renfro@gpunode009 build]$ cd ~/csc4760/kokkos
[renfro@gpunode009 kokkos]$ cd multi-gpu-cuda
[renfro@gpunode009 multi-gpu-cuda]$ mkdir build ; cd build
[renfro@gpunode009 build]$ cmake ..
-- The C compiler identification is GNU 14.3.0
...
[renfro@gpunode009 build]$ make
...
[100%] Built target MultiGpuCuda
```

## Example that actually uses a GPU

### Testing, releasing reservation

```
[renfro@gpunode009 build]$ ./MultiGpuCuda
...
Computed results for N=10000 and nrepeat=100 are
  1.0000000e+08 and 1.0000000e+08
N( 10000 ) nrepeat ( 100 ) problem( 1600.32 MB )
  time( 0.620942 s ) bandwidth( 515.449 GB/s )
[renfro@gpunode009 build]$ exit
exit
[renfro@login ~]$
```

# Thanks. Questions?



<https://www.rcd.tntech.edu/>, includes links to ITS site for documentation and access request forms, plus links for booking an appointment or getting to our Teams team.